MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

Relational vs NoSQL Databases

ACID vs BASE · Document · Key-Value · Graph — CAP Theorem

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:15	Week 5 recap poll	Review
1:15 – 1:50	ACID Properties & Relational Model Strengths	**Theory** 📖
1:50 – 2:15	NoSQL Landscape & BASE Properties	**Theory** 📖
2:15 – 2:30	☕ Break (10 min)	
2:30 – 2:55	CAP Theorem & Data Models	**Theory** 📖
2:55 – 3:15	SQL vs NoSQL: When to Use Which?	**Theory** 📖
3:15 – 3:40	Hands-on: Document-Style Queries in SQLite	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 5 Quick Recap

Concept	Key Point
View	Named SELECT query stored in the database (no data storage)
CREATE VIEW	`CREATE VIEW v AS SELECT ...`
Updatable view	Single table, no aggregation/DISTINCT/GROUP BY
WITH CHECK OPTION	Prevents INSERT/UPDATE that would be invisible to the view
Security views	Hide sensitive columns (salary, SSN) from certain users
INSTEAD OF trigger	Makes non-updatable views insertable/updatable
Materialized view	Stores results physically (not native in SQLite)
Best practice	Use `v_` prefix, keep nesting shallow

PART 1 — THEORY

The Relational Model & ACID

Relational Databases — The Foundation

Key characteristics:

- **Structured data** in tables (rows × columns)
- **Schema-first**: Define structure BEFORE inserting data
- **SQL** as the standard query language
- **Relationships** via foreign keys
- **Normalization** to reduce redundancy
- **ACID transactions** for data integrity

Popular Relational Databases:

Database	License	Use Case
SQLite	Public domain	Embedded, mobile, small apps
MySQL	Open source (Oracle)	Web apps (WordPress, etc.)
PostgreSQL	Open source	Complex apps, GIS, analytics
Oracle	Commercial	Enterprise
SQL Server	Commercial	Enterprise (.NET ecosystem)

ACID Properties

ACID = the four guarantees of relational database transactions.

Property	Meaning	Example
Atomicity	All or nothing — entire transaction succeeds or fails	Bank transfer: debit AND credit both happen, or neither
Consistency	Data always valid — constraints enforced	FK violations prevented
Isolation	Concurrent transactions don't interfere	Two users updating same account
Durability	Once committed, data survives crashes	COMMIT writes to disk

ACID — Atomicity

"All or nothing"

```sql
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 500 -- Transfer $500 from Account A to Account B
WHERE account_id = 'A';

UPDATE accounts SET balance = balance + 500
WHERE account_id = 'B';

COMMIT;  -- Both succeed → permanent
-- OR
ROLLBACK;  -- Something went wrong → undo BOTH
```

> If the system crashes between the two UPDATEs:
>
> - Without atomicity: Account A loses $500, B never gets it 💀
> - With atomicity: Both changes are rolled back ✅

ACID — Consistency

"Data always valid"

```sql
-- Constraint: balance must be >= 0
-- Constraint: total money in system stays constant

BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 1000
WHERE account_id = 'A';
-- If A only has $500 → CHECK constraint violation → ROLLBACK
COMMIT;
```

Consistency means:

- ✅ All constraints are enforced (PK, FK, CHECK, UNIQUE, NOT NULL)
- ✅ Data moves from one valid state to another
- ✅ Invalid transactions are rejected

ACID — Isolation

"Transactions don't interfere with each other"

```
Time   |  Transaction 1          |  Transaction 2
------+-------------------------+-------------------------
  1    |  Read balance: $1000    |
  2    |                         |  Read balance: $1000
  3    |  Withdraw $300          |
  4    |  Balance = $700         |  Withdraw $500
  5    |  COMMIT                 |  Balance = $500
  6    |                         |  COMMIT

Without isolation: Final balance = $500 (T2 overwrites T1) 💀
With isolation: T2 waits for T1 → reads $700 → final = $200 ✅
```

Isolation levels (from least to most strict):

1. **READ UNCOMMITTED** — dirty reads possible
2. **READ COMMITTED** — no dirty reads
3. **REPEATABLE READ** — same read = same result
4. **SERIALIZABLE** — full isolation (slowest)

ACID — Durability

"Once committed, it stays committed"

```
BEGIN TRANSACTION;
INSERT INTO orders (customer_id, total) VALUES (42, 99.99);
COMMIT;  -- Written to disk/WAL

-- Server crashes 1 millisecond later...
-- After restart: the order is still there ✅
```

How durability works:

- **Write-Ahead Logging (WAL)**: Changes logged before applied
- **Flush to disk**: COMMIT forces write to non-volatile storage
- **Recovery**: On crash restart, replay the WAL to restore state

🧠 Exam Focus: ACID

> **Q1:** Which ACID property ensures that a transaction is either fully completed or fully rolled back?
>
> A) Consistency
> B) **Atomicity** ✅
> C) Isolation
> D) Durability

> **Q2:** Which ACID property guarantees that once a transaction is committed, it will survive system failures?
>
> A) Atomicity
> B) Consistency
> C) Isolation
> D) **Durability** ✅

The NoSQL Landscape

Why NoSQL?

Limitations of relational databases at scale:

Challenge	Relational Struggle
Volume	Billions of rows → JOINs become expensive
Velocity	Millions of writes/sec → ACID overhead
Variety	Unstructured data (JSON, images, logs) → rigid schema
Horizontal scaling	Splitting across servers is hard (JOINs across machines?)

NoSQL = "Not Only SQL"

- Relaxes some relational constraints for **scalability** and **flexibility**
- Each type optimized for specific access patterns
- Often sacrifice consistency for availability (BASE)

BASE Properties

BASE = the NoSQL alternative to ACID.

Property	Meaning
Basically **A**vailable	System always responds, even if data is stale
Soft state	Data may change over time without explicit input (replication lag)
Eventual consistency	All replicas will eventually have the same data

ACID vs BASE:

	ACID	BASE
Consistency	Strong (immediate)	Eventual
Availability	May block (locking)	Always available
Best for	Banking, inventory	Social media, caching
Trade-off	Consistency over speed	Speed over consistency

Googles DBs: https://www.youtube.com/watch?v=R0XlH8FUEc8

Exam Focus: BASE

Q3: What does BASE stand for in the context of NoSQL databases?

A) Binary Access Storage Engine

B) **Basically Available, Soft state, Eventual consistency** ✅

C) Basic Availability Standard Extension

D) Batch Access System Environment

Q4: Which statement about BASE is TRUE?

A) BASE guarantees immediate consistency like ACID

B) **BASE systems prioritize availability over strict consistency** ✅

C) BASE is only used in relational databases

D) BASE means the database is not reliable

Types of NoSQL Databases

Type	Data Model	Examples	Best For
Document	JSON/BSON documents	MongoDB, CouchDB	Content management, catalogs
Key-Value	Simple key→value pairs	Redis, DynamoDB	Caching, sessions
Column-Family	Wide columns	Apache Cassandra, HBase	Time-series, IoT
Graph	Nodes + edges	Neo4j, Amazon Neptune	Social networks, recommendations

Document Databases (MongoDB)

Data stored as JSON/BSON documents:

```
{
    "_id": "ObjectId('6123...')",
    "name": "Ahmet Yılmaz",
    "email": "ahmet@ytu.edu.tr",
    "department": "Computer Science",
    "courses": [
        {"name": "Database Systems", "grade": "A"},
        {"name": "Algorithms", "grade": "B+"}
    ],
    "address": {
        "city": "Istanbul",
        "district": "Beşiktaş"
    }
}
```

Key features:

- **Schema-less**: Each document can have different fields
- **Nested data**: Arrays and objects inside documents
- **No JOINs**: Related data embedded in the same document
- **Horizontal scaling**: Easy to shard across servers

Key-Value Stores (Redis)

Simplest NoSQL model: key → value

```
SET user:42:name "Ahmet Yılmaz"
GET user:42:name → "Ahmet Yılmaz"

SET session:abc123 '{"userId":42,"role":"student"}'
GET session:abc123 → {"userId":42,"role":"student"}

SET product:101:stock 50
DECR product:101:stock → 49
```

Use cases:

- **Caching**: Store frequently accessed data in memory
- **Sessions**: Web application session management
- **Counters**: Real-time view counts, likes
- **Queues**: Task/message queues

Column-Family Stores (Cassandra)

Data organized by columns, not rows:

```
Row Key: "student:42"
  Column Family: "info"
    name: "Ahmet"     email: "ahmet@ytu.edu.tr"
  Column Family: "grades"
    CS101: "A"         CS201: "B+"         MATH301: "A-"
```

Key features:

- Columns can be added per-row (sparse data)
- Excellent for **time-series** data (IoT sensors, logs)
- Scales horizontally across many nodes

Use cases: IoT data, event logging, time-series analytics

Graph Databases (Neo4j)

Data as nodes and edges (relationships):

```
(Ahmet)-[:ENROLLED_IN]->(CS101)
(Ahmet)-[:FRIENDS_WITH]->(Mehmet)
(CS101)-[:TAUGHT_BY]->(Dr. Kaya)
(Dr. Kaya)-[:WORKS_IN]->(CS Department)
```

Cypher Query (Neo4j):

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
WHERE s.name = "Ahmet" RETURN c.name
```

Use cases:

- **Social networks**: Friend recommendations
- **Fraud detection**: Unusual transaction patterns
- **Knowledge graphs**: Wikipedia, Google Knowledge Graph
- **Recommendation engines**: "Users who bought X also bought Y"

PART 3 — THEORY

CAP Theorem

CAP Theorem (Brewer's Theorem, 2000)

```
        Consistency
           /\
          /  \
         /    \
        / CA   \  CP
       /________\
  Availability - Partition
                 Tolerance
```

Property	Meaning
Consistency	Every read returns the most recent write
Availability	Every request gets a response (no timeout)
Partition Tolerance	System works despite network failures between nodes

> ⚡ In a distributed system, **network partitions WILL happen**, so you must choose: Consistency OR Availability.

CAP — Practical Choices

Choice	Sacrifices	Systems	Use Case
CP (Consistent + Partition tolerant)	Availability	MongoDB, HBase, Redis	Banking, inventory
AP (Available + Partition tolerant)	Consistency	Cassandra, DynamoDB, CouchDB	Social media, caching
CA (Consistent + Available)	Partition tolerance	Traditional RDBMS (single node)	Not distributed

Real-world examples:

- **Banking app (CP)**: Better to show "system unavailable" than incorrect balance
- **Social media feed (AP)**: OK to show slightly stale posts rather than error page
- **SQLite (CA)**: Single-file, no network partitions to worry about

🧠 Exam Focus: CAP Theorem

Q5: According to the CAP theorem, a distributed database can guarantee at most how many of the three properties (Consistency, Availability, Partition Tolerance)?

A) All three simultaneously
B) **Two out of three** ✅
C) Only one at a time
D) None — it's a theoretical limit only

Q6: A system that prioritizes Availability and Partition Tolerance (AP) will sacrifice:

A) Performance
B) **Strong consistency** ✅
C) Data storage
D) Security

☕ BREAK (10 minutes)

SQL vs NoSQL: When to Use Which?

SQL vs NoSQL Comparison

Feature	SQL (Relational)	NoSQL
Schema	Fixed, predefined	Flexible, schema-less
Data model	Tables (rows × columns)	Documents, key-value, graphs
Query language	SQL (standardized)	Varies by database
Relationships	JOINs via foreign keys	Embedded/denormalized
Scaling	Vertical (bigger server/worker)	Horizontal (more servers/workers)
Consistency	Strong (ACID)	Eventual (BASE)
Best for	Structured data, complex queries	Unstructured data, high scale

When to Use Relational (SQL)

✅ **Choose SQL when:**

- Data has **clear structure** and relationships
- Need **complex queries** with JOINs, GROUP BY, subqueries
- **Data integrity** is critical (banking, healthcare, inventory)
- **ACID** compliance is required
- Data volume is manageable (< billions of rows)
- Need **ad-hoc reporting** and analytics

Examples:

- 🏦 Banking systems → account balances must be exact
- 🏥 Healthcare records → patient data must be consistent
- 📦 E-commerce orders → inventory must be accurate
- 🎓 University systems → enrollment, grades, transcripts

When to Use NoSQL

✅ **Choose NoSQL when:**

- Data is **unstructured** or **semi-structured** (JSON, logs)
- Need **horizontal scaling** (millions of users globally)
- **High write throughput** required
- Schema **changes frequently**
- **Availability** is more important than strong consistency
- Specific access patterns (key lookups, graph traversals)

Examples:

- 📱 Social media → billions of posts, flexible schema
- 🎮 Gaming → real-time leaderboards, session data
- 📊 IoT → sensor data from millions of devices
- 🛒 Product catalogs → varying attributes per product
- 💬 Chat apps → high write volume, eventual consistency OK

Polyglot Persistence

Modern applications often use BOTH SQL and NoSQL:

```
E-Commerce Platform:

├── PostgreSQL     → Orders, payments, inventory (ACID)
├── MongoDB        → Product catalog (flexible schema)
├── Redis          → Shopping cart, sessions (fast cache)
├── Elasticsearch  → Product search (full-text)
└── Neo4j          → Recommendations (graph)
```

> 💡 **Polyglot persistence** = using the right database for each job.

SQL in the NoSQL World

Many NoSQL databases now support SQL-like queries:

Database	SQL Support
MongoDB	Aggregation pipeline (SQL-like)
Cassandra	**CQL** (Cassandra Query Language)
Google BigQuery	Standard SQL
Amazon Athena	Standard SQL
Apache Spark	Spark SQL
Snowflake	Standard SQL

> The trend: SQL as a **universal query interface**, even for non-relational data.

🧠 Exam Focus: SQL vs NoSQL

> **Q7:** When would you choose a relational database over NoSQL?
>
> A) When you need horizontal scaling across 100 servers
>
> B) When data has no fixed schema
>
> C) **When data integrity and complex JOINs are critical** ✅
>
> D) When write throughput exceeds 1 million ops/sec

🧠 Exam Focus: Polyglot Persistence

Q8: What is "polyglot persistence"?

A) Using only one database for all data

B) **Using different database types for different data needs within one system** ✅

C) Translating SQL queries to multiple languages

D) A backup strategy using multiple servers

Normalization vs Denormalization

Relational (Normalized):

```
student table:  {student_id, name, dept_id}
department:     {dept_id, dept_name}
enrollment:     {student_id, course_id, grade}
course:         {course_id, course_name}
→ 3 JOINs needed for a transcript
```

NoSQL (Denormalized):

```
{ "name": "Ahmet Yılmaz",
    "department": "Computer Science",
    "courses": [
        {"name": "DB Systems", "grade": "A"},
        {"name": "Algorithms", "grade": "B+"}
    ] }
→ Single document read, no JOINs!
```

SQLite's JSON Support

```sql
-- Store JSON data in a TEXT column
CREATE TABLE products (
    id INTEGER PRIMARY KEY, name TEXT,
    attributes TEXT  -- JSON data
);

INSERT INTO products (name, attributes)
VALUES ('Laptop', '{"brand":"Dell","ram":16,"ssd":512}');

-- Query JSON fields
SELECT name, json_extract(attributes, '$.brand') AS brand FROM products;

-- Filter by JSON value
SELECT * FROM products WHERE json_extract(attributes, '$.ram') >= 16;
```

> 💡 SQLite gives you the flexibility of document-style data within a relational framework.

PART 5 — CODING PRACTICE

JSON & Document-Style Queries in SQLite

Practice: JSON in SQLite

```sql
-- Create a flexible product catalog
CREATE TABLE IF NOT EXISTS product_catalog (
    product_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    category TEXT NOT NULL,
    base_price REAL NOT NULL,
    specs TEXT  -- JSON for varying attributes
);

INSERT INTO product_catalog VALUES
(1, 'MacBook Pro 14', 'Laptop', 2499.00,
 '{"brand":"Apple","ram":16,"storage":512,"cpu":"M3","screen":14.2}'),
(2, 'ThinkPad X1', 'Laptop', 1799.00,
 '{"brand":"Lenovo","ram":32,"storage":1024,"cpu":"i7","screen":14}'),
(3, 'iPhone 15', 'Phone', 999.00,
 '{"brand":"Apple","storage":256,"color":"Blue","5g":true}'),
(4, 'Galaxy S24', 'Phone', 899.00,
 '{"brand":"Samsung","storage":128,"color":"Black","5g":true}'),
(5, 'iPad Air', 'Tablet', 599.00,
 '{"brand":"Apple","storage":64,"screen":10.9,"cellular":false}');
```

Practice: Querying JSON Data

```sql
-- Extract specific JSON fields
SELECT name,
       json_extract(specs, '$.brand') AS brand,
       json_extract(specs, '$.ram') AS ram_gb,
       json_extract(specs, '$.storage') AS storage_gb
FROM product_catalog
WHERE category = 'Laptop';

-- Filter by JSON values
SELECT name, base_price
FROM product_catalog
WHERE json_extract(specs, '$.brand') = 'Apple';

-- Range query on JSON numeric values
SELECT name, json_extract(specs, '$.ram') AS ram
FROM product_catalog
WHERE json_extract(specs, '$.ram') >= 16;

-- Check for JSON key existence
SELECT name, json_extract(specs, '$.5g') AS has_5g
FROM product_catalog
WHERE json_extract(specs, '$.5g') = 1;
```

Practice: JSON Modification

```sql
-- Update a JSON value
UPDATE product_catalog
SET specs = json_set(specs, '$.ram', 64)
WHERE product_id = 2;

-- Add a new JSON field
UPDATE product_catalog
SET specs = json_insert(specs, '$.warranty', '2 years')
WHERE product_id = 1;

-- Remove a JSON field
UPDATE product_catalog
SET specs = json_remove(specs, '$.color')
WHERE product_id = 3;

-- Verify changes
SELECT name, json_extract(specs, '$.ram') AS ram,
       json_extract(specs, '$.warranty') AS warranty
FROM product_catalog WHERE product_id IN (1, 2);

-- Pretty print JSON
SELECT name, json(specs) FROM product_catalog LIMIT 2;
```

Practice: JSON Arrays

```sql
-- Products with tags as JSON array
CREATE TABLE IF NOT EXISTS items (
    id INTEGER PRIMARY KEY,
    name TEXT,
    tags TEXT  -- JSON array
);

INSERT INTO items VALUES
(1, 'Gaming Laptop', '["gaming","laptop","high-performance"]'),
(2, 'Business Laptop', '["business","laptop","portable"]'),
(3, 'Gaming Mouse', '["gaming","mouse","rgb"]');

-- Check if array contains a value
SELECT name FROM items
WHERE EXISTS (
    SELECT 1 FROM json_each(tags) WHERE value = 'gaming'
);

-- Count tags per item
SELECT name,
       json_array_length(tags) AS tag_count
FROM items;

-- Expand JSON array (one row per tag)
SELECT i.name, j.value AS tag
FROM items i, json_each(i.tags) j;
```

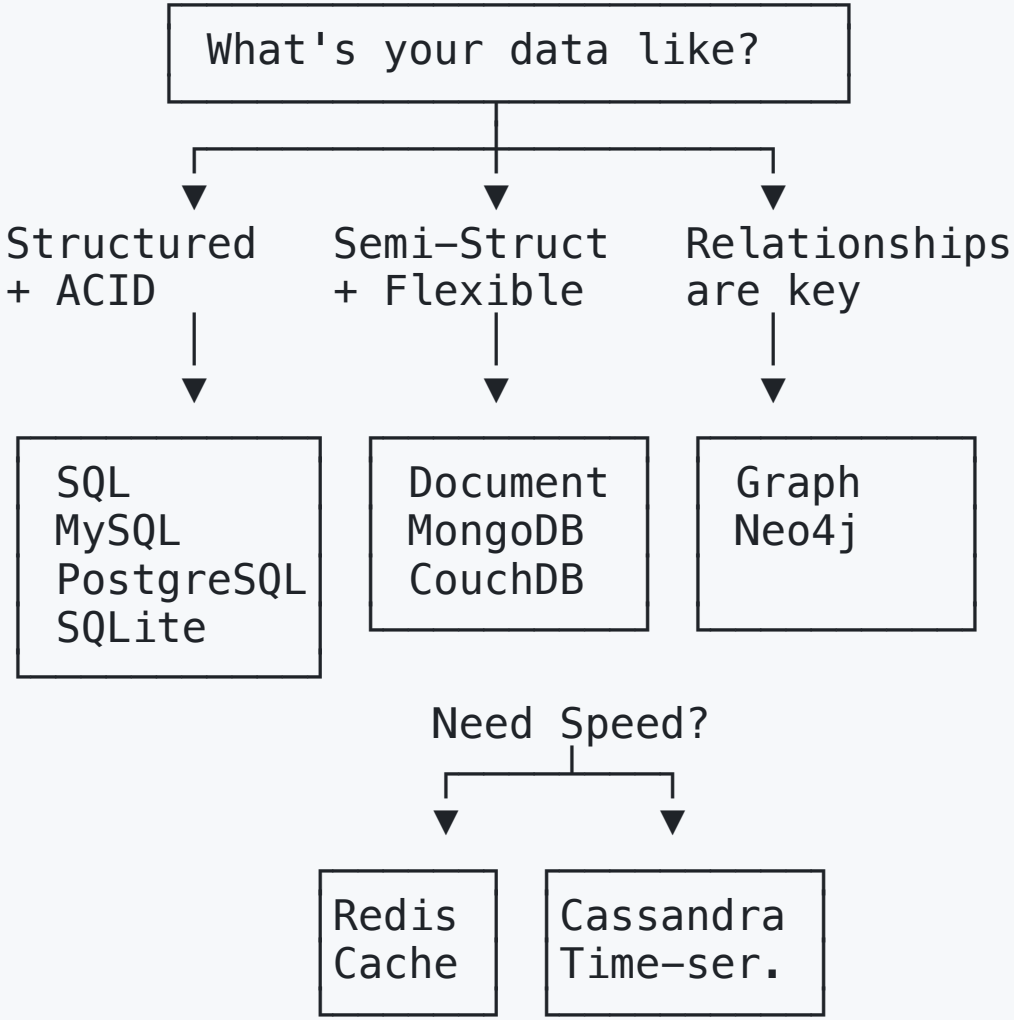
Practice: Hybrid SQL + JSON Reporting

```sql
-- Combine traditional SQL with JSON queries
SELECT category,
       COUNT(*) AS product_count,
       ROUND(AVG(base_price), 2) AS avg_price,
       GROUP_CONCAT(DISTINCT json_extract(specs, '$.brand')) AS brands
FROM product_catalog
GROUP BY category;

-- Price comparison by brand (from JSON)
SELECT json_extract(specs, '$.brand') AS brand,
       MIN(base_price) AS min_price,
       MAX(base_price) AS max_price,
       ROUND(AVG(base_price), 2) AS avg_price
FROM product_catalog
GROUP BY json_extract(specs, '$.brand');

-- Laptops sorted by RAM (from JSON)
SELECT name, base_price,
       json_extract(specs, '$.ram') AS ram_gb
FROM product_catalog
WHERE category = 'Laptop'
ORDER BY json_extract(specs, '$.ram') DESC;
```

Practice: ACID in SQLite

```sql
-- Demonstrate ACID properties in SQLite

-- Atomicity: all or nothing
BEGIN TRANSACTION;
INSERT INTO student (first_name, last_name, dept_id, gpa)
VALUES ('Test1', 'ACID', 1, 3.0);
INSERT INTO student (first_name, last_name, dept_id, gpa)
VALUES ('Test2', 'ACID', 1, 3.5);
ROLLBACK;  -- Neither insert is saved

SELECT COUNT(*) FROM student WHERE last_name = 'ACID';
-- Should be 0

-- Durability: committed data persists
BEGIN TRANSACTION;
INSERT INTO student (first_name, last_name, dept_id, gpa)
VALUES ('Durable', 'Test', 1, 3.0);
COMMIT;  -- Saved permanently

SELECT * FROM student WHERE first_name = 'Durable';
-- Row exists even if we close and reopen the database

-- Cleanup
DELETE FROM student WHERE first_name = 'Durable';
```

Database Selection Guide



Summary: Key Takeaways

Concept	Key Point
ACID	Atomicity, Consistency, Isolation, Durability — relational guarantee
BASE	Basically Available, Soft state, Eventual consistency — NoSQL approach
CAP Theorem	Distributed systems: pick 2 of Consistency, Availability, Partition Tolerance
Document DB	JSON documents, flexible schema (MongoDB)
Key-Value	Simple key→value, ultra-fast (Redis)
Column-Family	Wide columns, write-heavy (Cassandra)
Graph DB	Nodes + edges, relationship queries (Neo4j)
Polyglot	Use the right database for each job
SQLite JSON	`json_extract()`, `json_set()`, `json_each()`

Practice Exercises (Homework)

1. **ACID demonstration:**
   - Write SQLite code demonstrating Atomicity (BEGIN/ROLLBACK)
   - Explain how SQLite's WAL journal mode supports Durability
2. **JSON practice:**
   - Create an "events" table storing event details as JSON
   - Write queries using `json_extract`, `json_set`, `json_each`
   - Build a report combining SQL aggregation with JSON field extraction
3. **Essay (exam prep):**
   - Explain the CAP theorem with a real-world example
   - For your semester project, would SQL or NoSQL be better? Justify.

Next Week Preview

Week 7: Window Functions & CTEs

- ROW_NUMBER, RANK, DENSE_RANK, NTILE
- OVER(PARTITION BY ... ORDER BY ...)
- Running totals, moving averages
- Common Table Expressions (WITH clause)
- Recursive CTEs — hierarchical data

Preparation:

- 📖 Read: *Learning SQL* Ch. 16 (Analytic Functions)
- 💻 Complete JSON exercises
- 🎓 Watch: CS50 SQL — Optimizing

Thank You!

Questions?